

Julia Fractals

...

By: Jake VanZyverden

The Squaring map

$$Q_0(z) = z^2$$

When mapped to a unit circle this map is chaotic, and the basis for the julia set.

The Julia Set

$$Q_0(z) = z^2 + c$$

Adding a complex number as a constant to the squaring map we create the julia set.

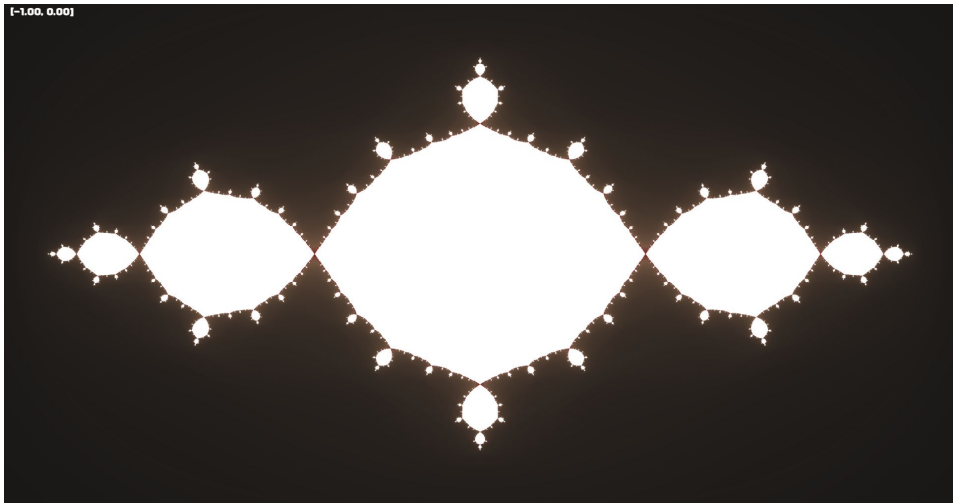
The Julia Set

$$Q_0(z) = z^2 + c$$

When mapping the julia set where $c \neq 0, -2$ we see self similar fractal like designs.

The Julia Set

$$Q_0(z) = z^2 + c$$



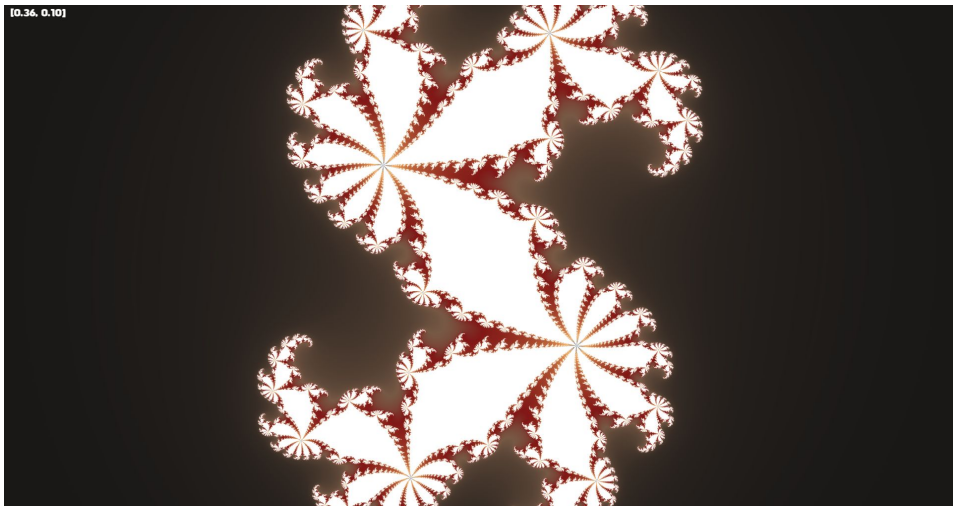
$C = -1.0 + 0.0i$ ($max_iter = 500$)

—

The Julia Set

$$Q_0(z) = z^2 + c$$

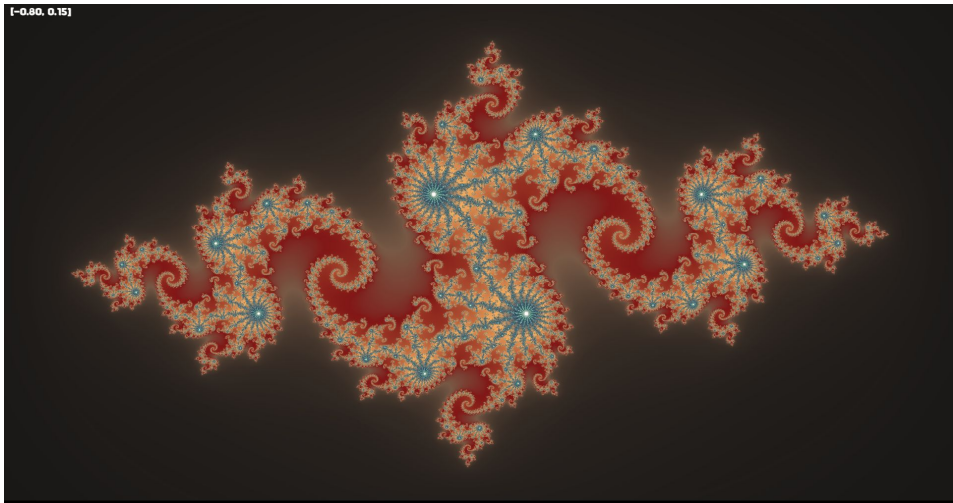
[0.36, 0.10]



$$C = 0.360284 + 0.100376i \text{ (max_iter = 250)}$$

The Julia Set

$$Q_0(z) = z^2 + c$$



$$C = -0.8 + 0.15i \text{ (max_iter = 500)}$$

—

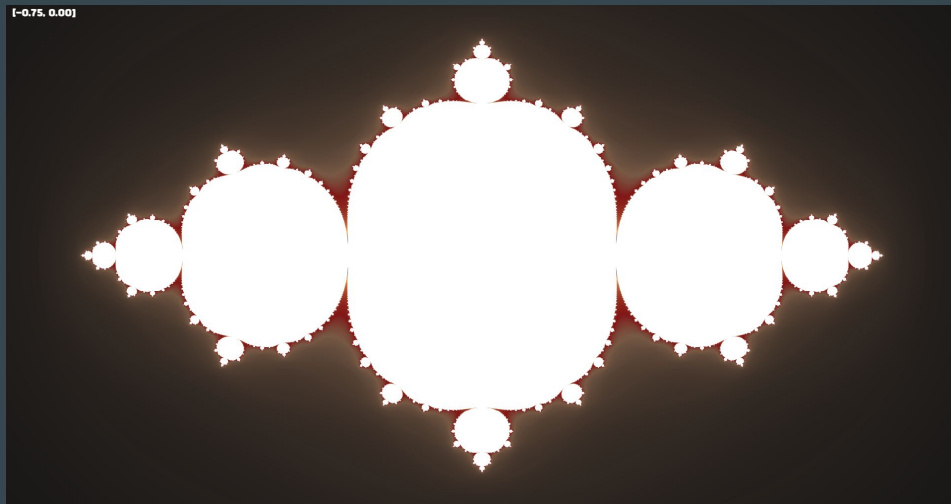
Properties of the Julia Set (Critical Orbits)

- Bounded & Connected

- Unbounded & Shattered

Properties of the Julia Set (Critical Orbits)

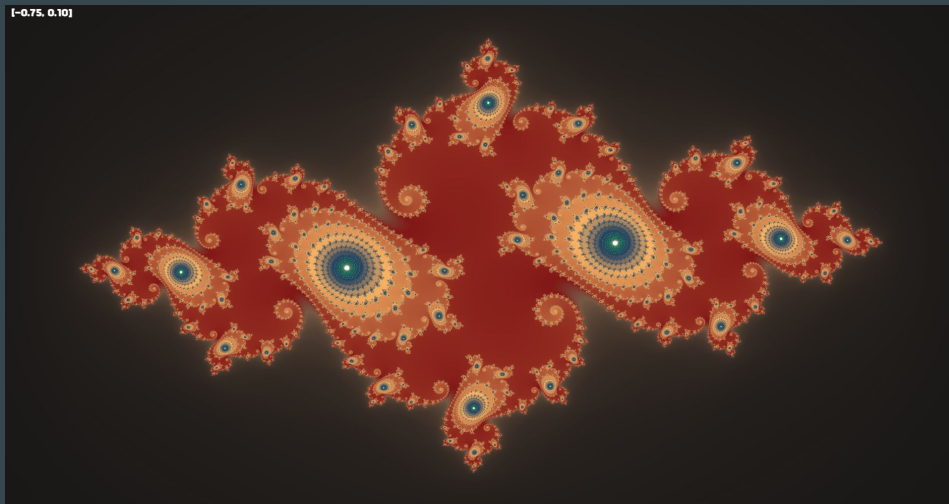
- **Bounded & Connected**
 - Orbit of 0 is bounded within the frame and cannot escape to infinity.
- **Unbounded & Shattered**



$C = -0.75 + 0.0i$ ($max_iter = 250$)

Properties of the Julia Set (Critical Orbits)

- Bounded & Connected
- Unbounded & Shattered
 - Orbit of 0 escapes to infinity before max iterations.



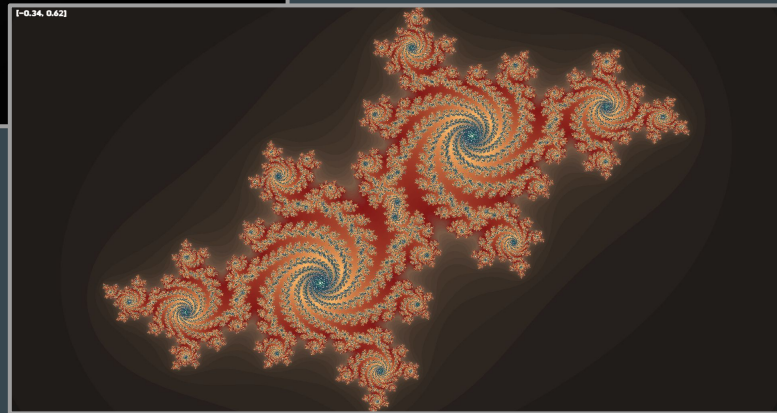
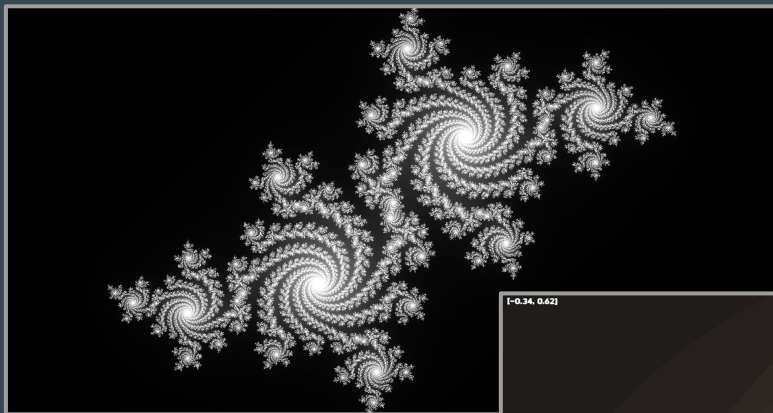
$$C = -0.75 + 0.1i \text{ (max_iter = 500)}$$

Code Review

- Gradients
- Julia iteration functions
 - Rough iterates
 - Smooth iterates
- Anti-aliasing
- Optimization

Code Review

- **Gradients**
- Julia iteration functions
 - Rough iterates
 - Smooth iterates
- Anti-aliasing
- Optimization



Code Review

- Gradients
- Julia iteration functions
 - Rough iterates
 - Smooth iterates
- Anti-aliasing
- Optimization

```
case Gradient::BLACK_AND_WHITE:  
    colorPoints.emplace_back( a1: 0.0f,  a2: sf::Vector3f{ X: 0.0f,  Y: 0.0f,  Z: 0.0f});  
    colorPoints.emplace_back( a1: 1.0f,  a2: sf::Vector3f{ X: 255.0f, Y: 255.0f, Z: 255.0f});  
    break;
```

```
case Gradient::DARK_MULTICOLOR:  
    colorPoints.emplace_back( a1: 0.0f,  a2: sf::Vector3f( X: 25,  Y: 24,  Z: 23));  
    colorPoints.emplace_back( a1: 0.03f, a2: sf::Vector3f( X: 120, Y: 90,  Z: 70));  
    colorPoints.emplace_back( a1: 0.05f, a2: sf::Vector3f( X: 130, Y: 24,  Z: 23));  
    colorPoints.emplace_back( a1: 0.25f, a2: sf::Vector3f( X: 250, Y: 179, Z: 100));  
    colorPoints.emplace_back( a1: 0.5f,  a2: sf::Vector3f( X: 43,  Y: 65,  Z: 98));  
    colorPoints.emplace_back( a1: 0.85f, a2: sf::Vector3f( X: 11,  Y: 110, Z: 79));  
    colorPoints.emplace_back( a1: 0.95f, a2: sf::Vector3f( X: 150, Y: 110, Z: 79));  
    colorPoints.emplace_back( a1: 1.0f,  a2: sf::Vector3f( X: 255, Y: 255, Z: 255));  
    break;
```

Code Review

- Gradients
- Julia iteration functions
 - Rough iterates
 - Smooth iterates
- Anti-aliasing
- Optimization

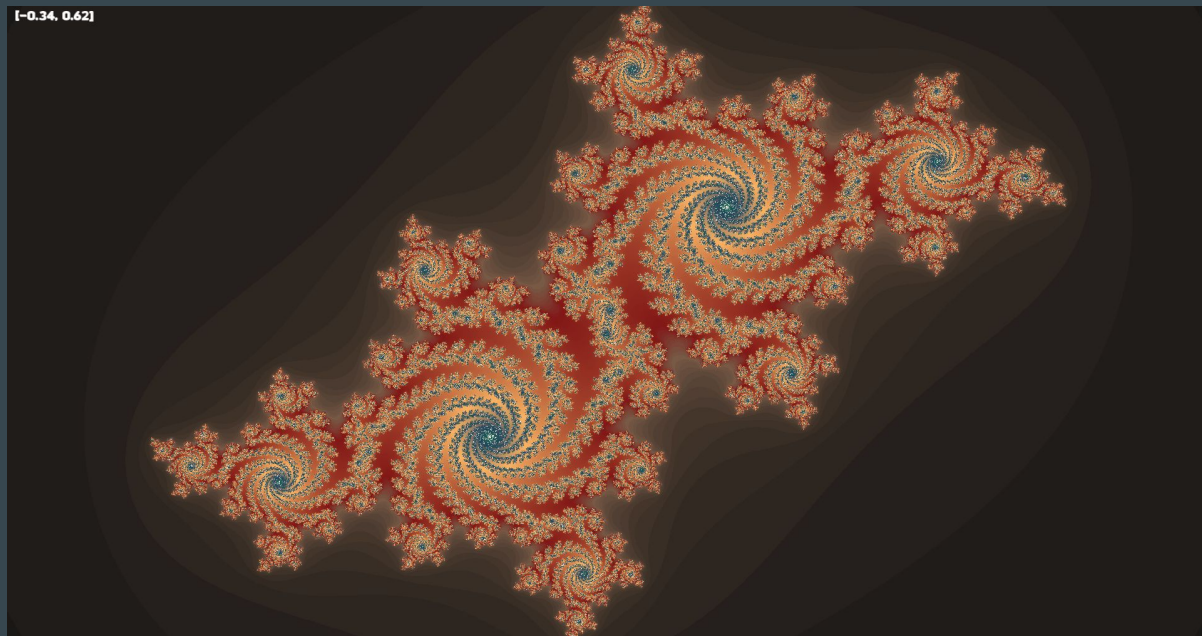
```
static sf::Vector3f applyGradient(float iter_ratio, Gradient gradient) {
    sf::Vector3f returnColor = sf::Vector3f{ X: 0.0f, Y: 0.0f, Z: 0.0f};

    std::vector<std::tuple<float, sf::Vector3f>> colorPoints = gradientMappings(gradient);
    if (iter_ratio ≥ 1.0f) {
        return std::get<1>( & colorPoints.back());
    } else if (iter_ratio ≤ 0.0f) {
        return std::get<1>( & colorPoints.front());
    }
    int i = 0;
    for (std::tuple<float, sf::Vector3f> point: colorPoints) {
        if (iter_ratio < std::get<0>( & point)) {
            sf::Vector3f currIndex = std::get<1>( & point);
            float range = std::get<0>( & point) - std::get<0>( & colorPoints[i - 1]);
            float pos = iter_ratio - std::get<0>( & colorPoints[i - 1]);
            float ratio = pos / range;

            float red = (1.0f - ratio) * std::get<1>( & colorPoints[i - 1]).x + ratio * currIndex.x;
            float green = (1.0f - ratio) * std::get<1>( & colorPoints[i - 1]).y + ratio * currIndex.y;
            float blue = (1.0f - ratio) * std::get<1>( & colorPoints[i - 1]).z + ratio * currIndex.z;
            returnColor = sf::Vector3f( X: red, Y: green, Z: blue);
            break;
        }
        ++i;
    }
    return returnColor;
}
```

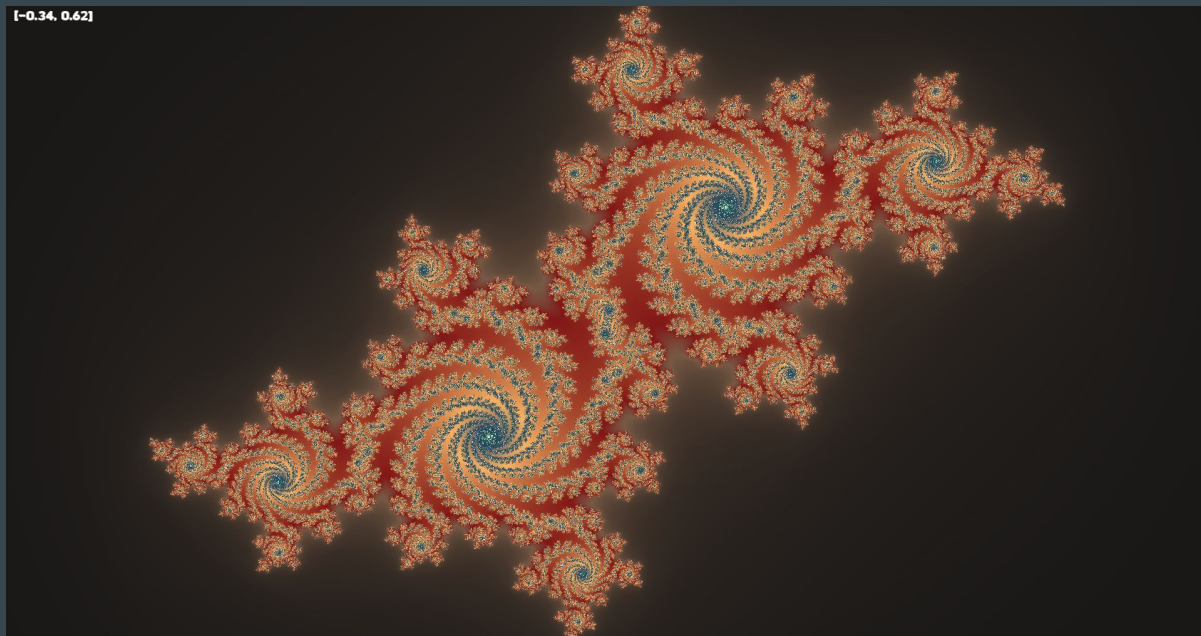
Code Review

- Gradients
- **Julia iteration functions**
 - Rough iterates
 - Smooth iterates
- Anti-aliasing
- Optimization



Code Review

- Gradients
- **Julia iteration functions**
 - Rough iterates
 - **Smooth iterates**
- Anti-aliasing
- Optimization



Code Review

- Gradients
- Julia iteration functions
 - Rough iterates
 - Smooth iterates
- Anti-aliasing
- Optimization

```
/**
 * iterates the given pixel coordinates (x,y) until either it reaches max iterations or escapes to infinity
 * @param zi (x,y) coordinates for initial conditions
 * const_c arbitrary complex number that the entire image is based on.
 * @return number of iterations computing took (used for gradient)
 */
float JuliaSet::iterate_coordinate(sf::Vector2f zi) {
    float zx = zi.x;
    float zy = zi.y;
    float mod = zx * zx + zy * zy;
    int curr_iter = 0;
    while (mod < 4.0f && (curr_iter < MAX_ITERATIONS)) {
        const float tmp = zx;
        // x coord = x^2 - y^2 + const_x
        zx = zx * zx - zy * zy + const_c.x;
        // y coord = 2.0 (radius of circle to fold) * y * x + const_y
        zy = 2.0f * zy * tmp + const_c.y;
        // modulus or norm of complex number is sqrt(x^2 + y^2), or simpler we check x^2 + y^2 < upper_bound^2 (2^2 = 4.0)
        mod = zx * zx + zy * zy;
        ++curr_iter;
    }
    return static_cast<float>(curr_iter);
}

const float final_mod = std::sqrt( x: mod);
const float smooth_iters = float(curr_iter) - std::log2( x: std::max(1.0f, std::log2( x: final_mod)));
return smooth_iters;
}
```

Code Review

- Gradients
- Julia iteration functions
 - Rough iterates
 - Smooth iterates
- **Anti-aliasing**
- Optimization

```
std::array<sf::Vector2f, 32> anti_aliasing_offsets;  
for (auto &o : Vector2<float> & : anti_aliasing_offsets) {  
    o.x = RNG( max: 1.0f);  
    o.y = RNG( max: 1.0f);  
    std::cout << o.x << "," << o.y << std::endl;  
}
```

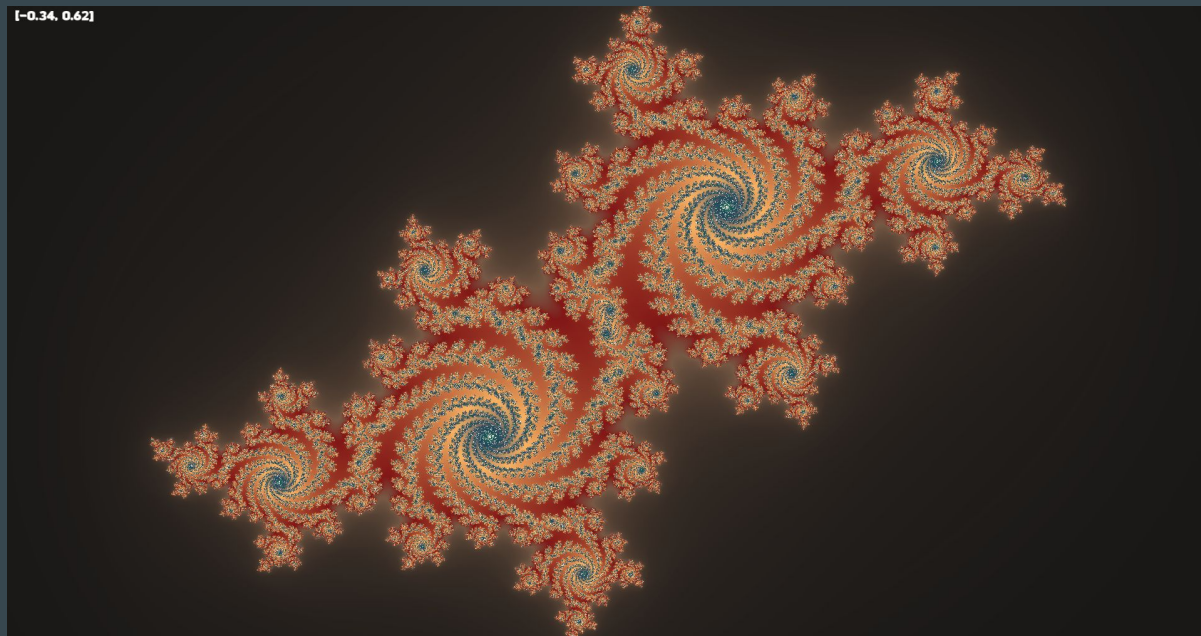
```
anti_aliasing_offsets[0] = {};
```

```
const int samples = 16;  
sf::Vertex vertex;  
sf::Vector3f colorSamples;  
for (int i{samples}; i--;) {
```

```
sf::Vector3f finalColor = colorSamples / float(samples);  
sf::Color color = Gradients::vec3ToColor( v: finalColor);  
for (int i{samples}; i--;) {
```

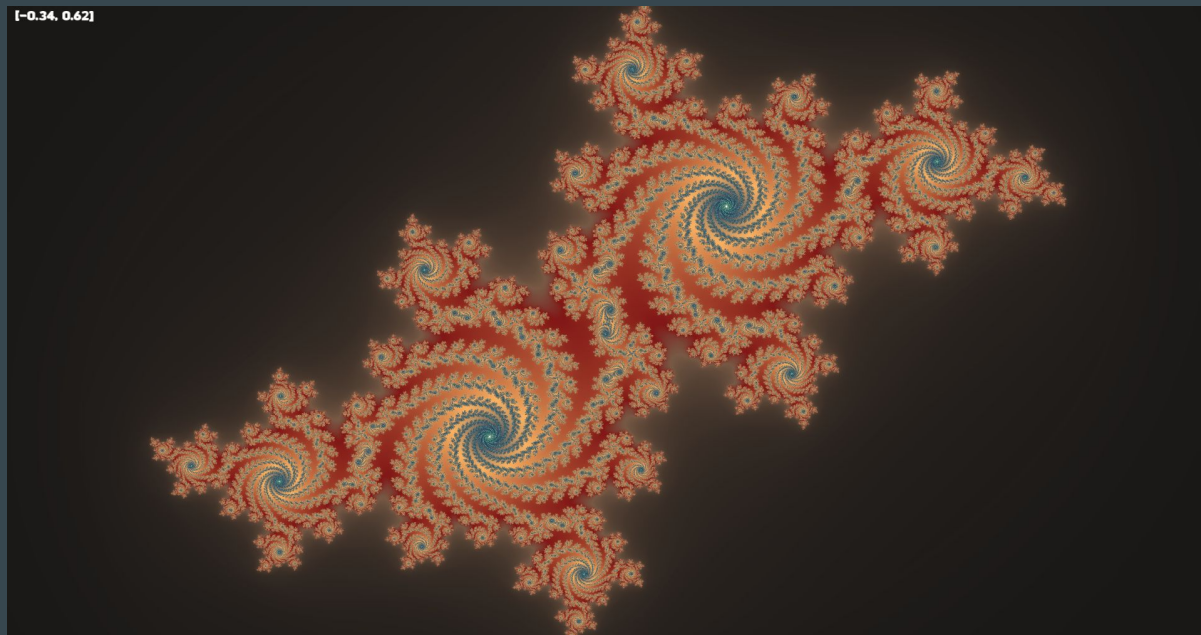
Code Review

- Gradients
- Julia iteration functions
 - Rough iterates
 - Smooth iterates
- **Anti-aliasing**
- Optimization



Code Review

- Gradients
- Julia iteration functions
 - Rough iterates
 - Smooth iterates
- **Anti-aliasing**
- Optimization



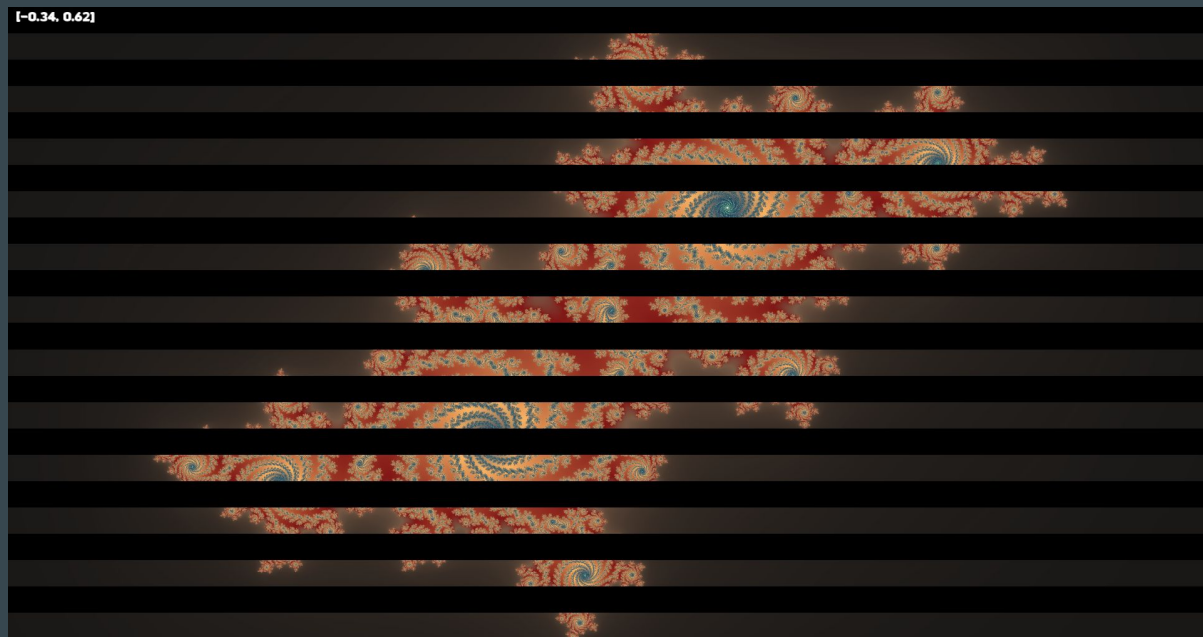
Code Review

- Gradients
- Julia iteration functions
 - Rough iterates
 - Smooth iterates
- Anti-aliasing
- Optimization

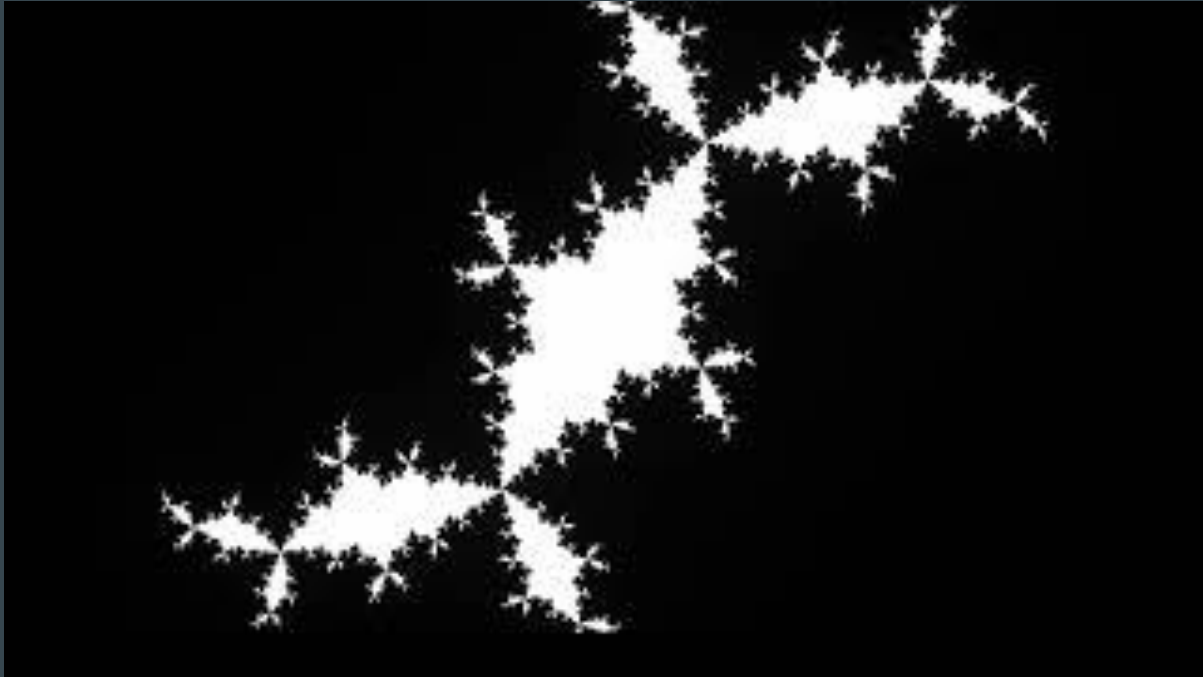
```
int thread_count = 24;
int slice_height = window_size.y / thread_count;
std::vector<std::thread> thread_pool;
for (int k{0}; k < thread_count; k++) {
    thread_pool.emplace_back( f, &Render::computeSlice, this, window_size, k, slice_height, anti_aliasing_offsets,
                             MAX_ITERATIONS, std::ref( & js), std::ref( & image));
}
for (std::thread &t: thread_pool) {
    t.join();
}
```

Code Review

- Gradients
- Julia iteration functions
 - Rough iterates
 - Smooth iterates
- Anti-aliasing
- **Optimization**

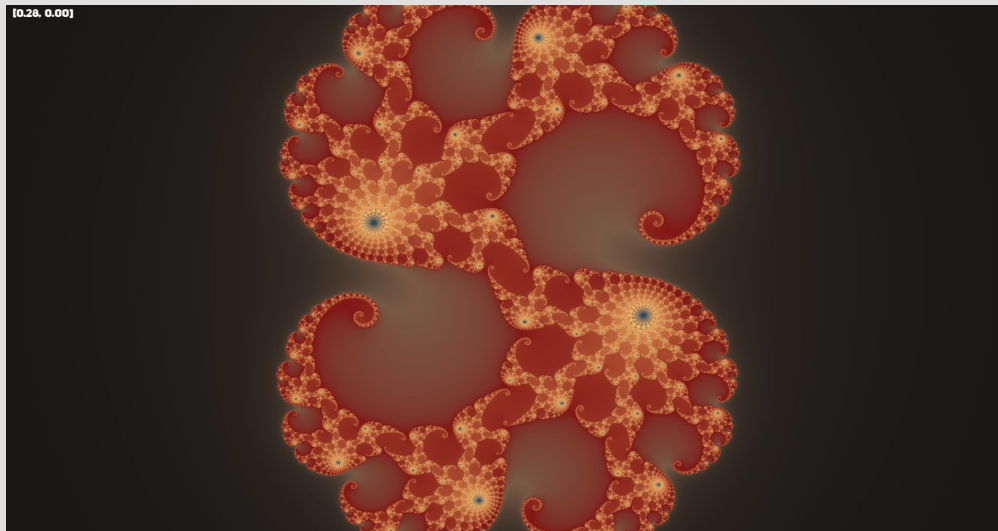


Video Demo $(-0.8, 0.0) \cup (0.8, 0.0)$



Thank You!

Questions?



$C = 0.28 + 0.008i$ ($max_iter = 500$)

Credits

- <https://github.com/johnBuffer/JuliaRendering>
- <https://www.sfml-dev.org/index.php>
- <https://www.karlsims.com/julia.html#:~:text=Julia%20set%20fractals%20are%20normally,gives%20a%20specific%20Julia%20set.>